



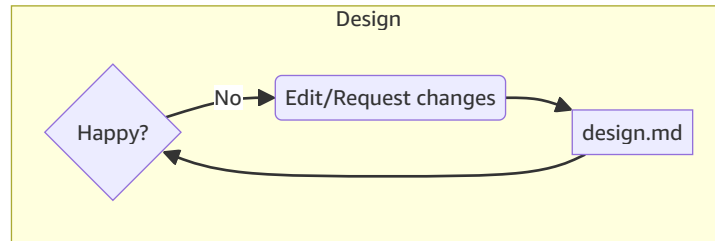
[Event dashboard](#) > [Create the application](#) > Produce design specifications

Produce design specifications

Overview

In this section you will produce a detailed [Kiro design](#), which will be specified in a `design.md` file.

The `design.md` file is where you document technical architecture, sequence diagrams, and implementation considerations. This file describes 'the big picture' of how the system will work, including the components and their interactions.



Examine the draft design

At the end of the last section, you moved to the design phase. Kiro has produced a `design.md` document. Select the **Design** icon to open the file.

```
1 # Design Document: Flappy Kiro
2
3 ## Overview
4
5 Flappy Kiro is a browser-based endless scroller game built with vanilla JavaScript and the HTML5
6 Canvas API. The player controls a ghost character ("Ghosty") navigating through gaps between pairs of
7 scrolling pipes. The game features a hand-drawn retro aesthetic, physics-based movement, and
8 persistent high score tracking.
9
10 The architecture follows a classic game loop pattern with a finite state machine managing game phases
11 (idle, active, game-over). All rendering is done through the Canvas 2D context, with entity
12 management handled by simple arrays rather than a full ECS framework – appropriate for the small
13 entity count.
14
15 **Key design decisions:**
16 - Single HTML file with embedded or linked JS for zero-build simplicity
17 - `requestAnimationFrame`-based game loop with delta-time awareness
18 - State machine pattern for clean phase transitions
19 - Pure functions for collision detection and physics to enable testability
20 - localStorage for high score persistence
21
22 ## Architecture
23
24 ```mermaid
25 graph TD
26   subgraph Entry
27     HTML[index.html] --> Init[Game.init]
28   end
```

Refine the design

After reviewing the `design.md` file, you can ask Kiro to enhance the design.

Ask Kiro for best practices

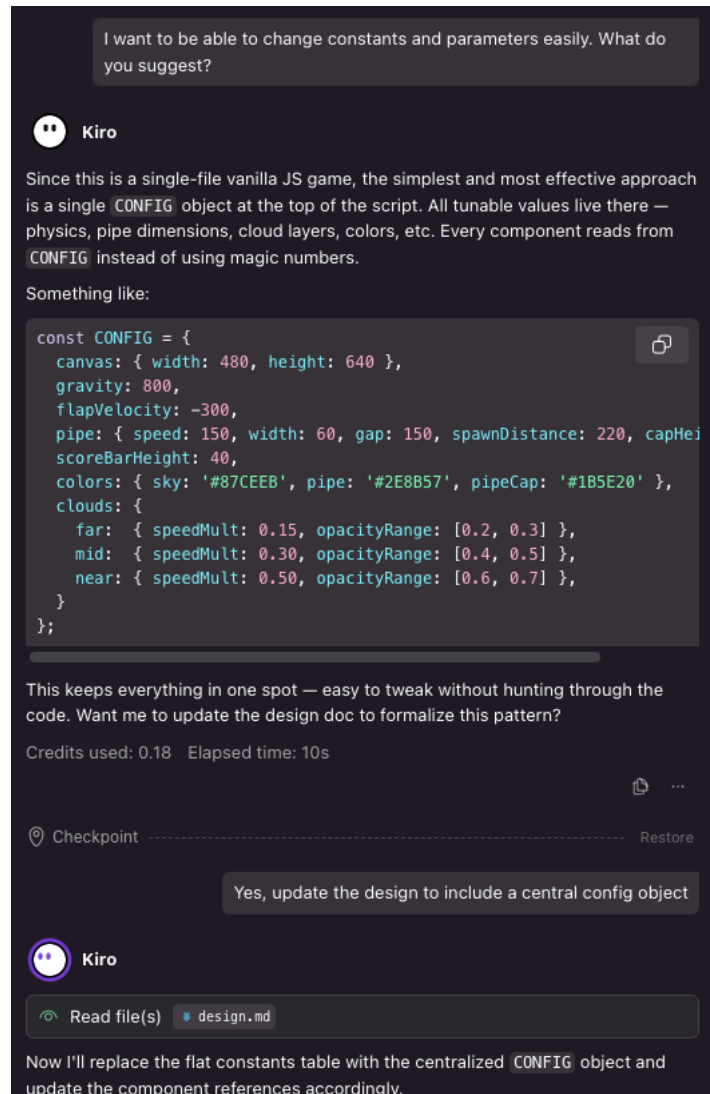
You may want to ask Kiro about considerations to get a better design.

1. Ask Kiro about where to store code parameters and constants.

► Example prompts

Update the design with a central location for application constants

1. Type a prompt instructing Kiro to centralize parameters.



(Optional) Add additional design features

Similar to the requirements phase, you can review and update your `design.md` file.

► Example prompts

(Optional) Correctness using property-based tests

To ensure that the generated application meets your expectations, [Kiro correctness](#) uses *Property-based testing* (PBT). This allows Kiro to validate universal properties across the entire project. By automatically translating natural language specifications into executable properties and generating comprehensive test cases, Kiro creates a powerful feedback loop that helps both AI agents and human developers build more reliable software.

During the design phase, Kiro extracts properties from your requirements and generates test cases. This is the first step in the workflow, where Kiro analyzes your requirements and identifies the properties that can be tested.

To examine the property based tests for your application, look for the `Correctness Properties` section in the `design.md` file.

```
flappy-kiro 1 Requirements > 2 Design > 3 Task list Sync Files

1 # Design Document: Flappy Kiro
230 ## Data Models
318 ### localStorage Schema
323
324
325 ## Correctness Properties
326
327 *A property is a characteristic or behavior that should hold true across all valid executions of a system – essentially,
a formal statement about what the system should do. Properties serve as the bridge between human-readable specifications
and machine-verifiable correctness guarantees.*
328
329 ### Property 1: Canvas scaling preserves aspect ratio
330
331 *For any* browser viewport dimensions (width, height), the scaled canvas dimensions should maintain the same aspect
ratio as the logical resolution (CANVAS_WIDTH / CANVAS_HEIGHT).
332
333 **Validates: Requirements 1.1**
334
335 ### Property 2: Gravity increases downward velocity each frame
336
337 *For any* initial vertical velocity and any positive deltaTime, after a physics update, Ghosty's velocity should equal
the initial velocity plus (GRAVITY × deltaTime).
338
339 **Validates: Requirements 2.1**
340
341 ### Property 3: Flap sets velocity to fixed upward value
342
343 *For any* current vertical velocity, after a flap is applied, Ghosty's velocity should equal the constant FLAP_VELOCITY
(a negative/upward value), regardless of the prior velocity.
344
345 **Validates: Requirements 2.2**
346
347 ### Property 4: Ghosty vertical position is clamped to canvas top
348
349 *For any* Ghosty position and velocity, after a physics update, Ghosty's y-coordinate should never be less than 0.
350
351 **Validates: Requirements 2.4**
352
353 ### Property 5: Pipes scroll at constant speed
354
355 *For any* set of active pipe pairs and any positive deltaTime, after an update each pipe's x-position should decrease by
exactly (PIPE_SPEED × deltaTime).
356
357 **Validates: Requirements 3.1**
358
```

Complete the design

1. Make a commit of design to your local git repository with the following command in your IDE terminal window:

```
git add .
git commit -m 'Produced design'
```



2. When you have completed the design, choose **Continue > Generate Tasks** at the top of the editor window to move to the next stage.

```
.kiro > specs > flappy-kiro > design.md > @ # Design Document: Flappy Kiro

flappy-kiro 1 Requirements > 2 Design > 3 Task list Sync Files Continue Generate Tasks

1 # Design Document: Flappy Kiro
3 ## Overview
physics, collision detection, scoring with localStorage persistence, parallax cloud backgrounds, and sound effects. The
entire game is delivered as a single 'index.html' file referencing three asset files.
6
7 The architecture follows a classic game loop pattern: a 'requestAnimationFrame'-driven loop that updates game state
(physics, pipe spawning, collision checks, scoring) and renders all entities to a 2D canvas each frame. The game
transitions between three states: 'start', 'playing', and 'game-over'.
8
9 ## Architecture
```

Tip

Kiro is flexible enough to allow you to have multiple ways to design and build your application. You can also simply instruct Kiro through the chat panel to move to the next stage, similar to what you did to move from the requirements to design stage.

Wrap up

You have completed the design document.

Next you will produce, review, iterate and complete the detailed implementation plan.

[Previous](#)

[Next](#)